

Лабораторная работа №5

Основы информационной безопасности

Краснова К. Г., НКАбд-03-24

17 апреля 2026

Российский университет дружбы народов, Москва, Россия

Изучение механизмов изменения идентификаторов, применения SetUID- и Sticky-битов.

Получение практических навыков работы в консоли с дополнительными атрибутами.

Рассмотрение работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

1. Дополнительные атрибуты файлов Linux

В Linux существует три основных вида прав — право на чтение (read), запись (write) и выполнение (execute), а также три категории пользователей, к которым они могут применяться — владелец файла (user), группа владельца (group) и все остальные (others). Но, кроме прав чтения, выполнения и записи, есть еще три дополнительных атрибута. (u?)

Sticky bit

Используется в основном для каталогов, чтобы защитить в них файлы. В такой каталог может писать любой пользователь. Но, из такой директории пользователь может удалить только те файлы, владельцем которых он является. Примером может служить директория `/tmp`, в которой запись открыта для всех пользователей, но нежелательно удаление чужих файлов.

SUID (Set User ID)

Атрибут исполняемого файла, позволяющий запустить его с правами владельца. В Linux приложение запускается с правами пользователя, запустившего указанное приложение. Это обеспечивает дополнительную безопасность т.к. процесс с правами пользователя не сможет получить доступ к важным системным файлам, которые принадлежат пользователю root.

SGID (Set Group ID)

Аналогичен `suid`, но относиться к группе. Если установить `sgid` для каталога, то все файлы созданные в нем, при запуске будут принимать идентификатор группы каталога, а не группы владельца, который создал файл в этом каталоге.

Обозначение атрибутов sticky, suid, sgid

Специальные права используются довольно редко, поэтому при выводе программы `ls -l` символ, обозначающий указанные атрибуты, закрывает символ стандартных прав доступа.

Пример: **rwsrwsrwt**

где первая `s` — это `suid`, вторая `s` — это `sgid`, а последняя `t` — это `sticky bit`

В приведенном примере не понятно, `rw` — это `rw`- или `rw``x`? Определить это просто. Если `t` маленькое, значит `x` установлен. Если `T` большое, значит `x` не установлен. То же самое правило распространяется и на `s`.

В числовом эквиваленте данные атрибуты определяются первым символом при четырехзначном обозначении (который часто опускается при назначении прав), например в правах `1777` — символ `1` обозначает `sticky bit`. Остальные атрибуты имеют следующие числовое соответствие:

`1` — установлен `sticky bit`

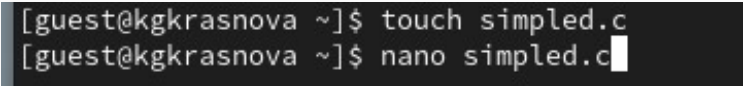
`2` — установлен `sgid`

`4` — установлен `suid`

2. Компилятор GCC

GCC - это свободно доступный оптимизирующий компилятор для языков C, C++. Собственно программа gcc это некоторая надстройка над группой компиляторов, которая способна анализировать имена файлов, передаваемые ей в качестве аргументов, и определять, какие действия необходимо выполнить. Файлы с расширением .cc или .C рассматриваются, как файлы на языке C++, файлы с расширением .c как программы на языке C, а файлы с расширением .o считаются объектными (gcc?).

Создание файла `simplified.c` и запись в файл кода (рис. 1).

A terminal window with a black background and white text. It shows two commands being executed in sequence. The first command is `[guest@kgkrasnova ~]$ touch simplified.c` and the second is `[guest@kgkrasnova ~]$ nano simplified.c`. A white cursor is visible at the end of the second command.

```
[guest@kgkrasnova ~]$ touch simplified.c  
[guest@kgkrasnova ~]$ nano simplified.c
```

Рис. 1: создание файла

Выполнение лабораторной работы

С++ Листинг 1 `#include <sys/types.h> #include <unistd.h> #include <stdio.h> int main () { uid_t uid = geteuid (); gid_t gid = getegid (); printf ("uid=%d, gid=%d\n", uid, gid); return 0; } ## Выполнение лабораторной работы`

Содержимое файла (рис. 2).



```
GNU nano 5.6.1 simplified.c Изменён
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
uid_t uid = geteuid ();
gid_t gid = getegid ();
printf ("uid=%d, gid=%d\n", uid, gid);
return 0;
}
```

Рис. 2: Содержимое файла

Компилирую файл, проверяю, что он скомпилировался (рис. 3).

```
[guest@kgkrasnova ~]$ gcc simplified.c -o simplified
[guest@kgkrasnova ~]$ ls
dir1  simplified  Видео      Загрузки    Музыка      'Рабочий стол'
dir2  simplified.c  Документы  Изображения  Общедоступные  Шаблоны
[guest@kgkrasnova ~]$
```

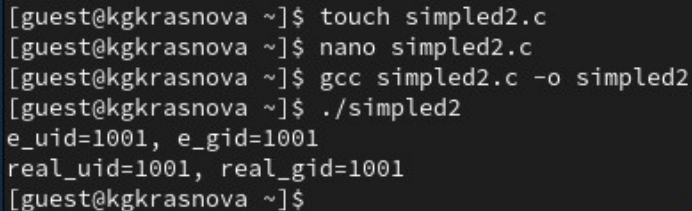
Рис. 3: Компиляция

Запускаю исполняемый файл. В выводе файла выписаны номера пользователя и групп (рис. 4).

```
[guest@kgkrasnova ~]$ ./simplified
uid=1001, gid=1001
[guest@kgkrasnova ~]$ id
uid=1001(guest) gid=1001(guest) группы=1001(guest) контекст=unconfined_u:unconfi
ned_r:unconfined_t:s0-s0:c0.c1023
[guest@kgkrasnova ~]$
```

Рис. 4: Запуск

Создание, запись в файл и компиляция файла `simplified2.c`. Запуск программы (рис. 5).

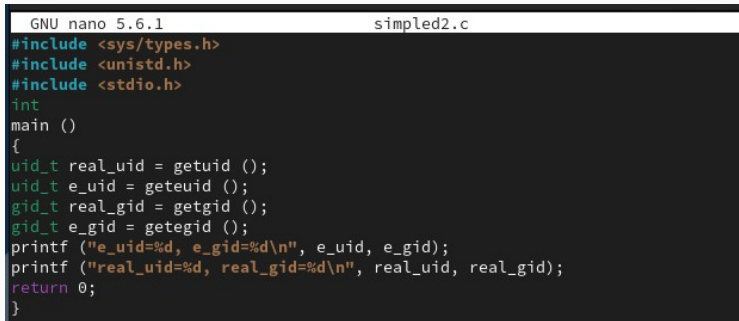
A terminal window with a dark background and light-colored text. The prompt is [guest@kgkrasnova ~]\$. The commands and their outputs are as follows:
1. touch simplified2.c
2. nano simplified2.c
3. gcc simplified2.c -o simplified2
4. ./simplified2
 e_uid=1001, e_gid=1001
 real_uid=1001, real_gid=1001
5. [guest@kgkrasnova ~]\$

```
[guest@kgkrasnova ~]$ touch simplified2.c
[guest@kgkrasnova ~]$ nano simplified2.c
[guest@kgkrasnova ~]$ gcc simplified2.c -o simplified2
[guest@kgkrasnova ~]$ ./simplified2
e_uid=1001, e_gid=1001
real_uid=1001, real_gid=1001
[guest@kgkrasnova ~]$
```

Рис. 5: Создание и запуск

```
С++ Листинг 2 #include <sys/types.h> #include <unistd.h> #include  
<stdio.h> int main () { uid_t real_uid = getuid (); uid_t e_uid =  
geteuid (); gid_t real_gid = getgid (); gid_t e_gid = getegid () ;  
printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid); printf ("real_uid=%d,  
real_gid=%d\n", real_uid, real_gid); return 0; }
```

Содержимое файла (рис. 6).



```
GNU nano 5.6.1                                simplified2.c
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
int
main ()
{
    uid_t real_uid = getuid ();
    uid_t e_uid = geteuid ();
    gid_t real_gid = getgid ();
    gid_t e_gid = getegid ();
    printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);
    printf ("real_uid=%d, real_gid=%d\n", real_uid, real_gid);
    return 0;
}
```

Рис. 6: Содержимое файла

С помощью `chown` изменяю владельца файла на суперпользователя, с помощью `chmod` изменяю права доступа (рис. 7).

```
[kgkrasnova@kgkrasnova ~]$ sudo chown root:guest /home/guest/simplified2
[sudo] пароль для kgkrasnova:
[kgkrasnova@kgkrasnova ~]$ sudo chmod u+s /home/guest/simplified2
[kgkrasnova@kgkrasnova ~]$ sudo ls -l /home/guest/simplified2
-rwsr-xr-x. 1 root guest 17656 фев 21 13:11 /home/guest/simplified2
[kgkrasnova@kgkrasnova ~]$
```

Рис. 7: Смена владельца

Сравнение вывода программы и команды `id` (рис. 8).

```
[kgkrasnova@kgkrasnova ~]$ sudo /home/guest/simplified2
e_uid=0, e_gid=0
real_uid=0, real_gid=0
[kgkrasnova@kgkrasnova ~]$ id
uid=1000(kgkrasnova) gid=1000(kgkrasnova) группы=1000(kgkrasnova),10(wheel) контекст=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
[kgkrasnova@kgkrasnova ~]$ sudo id
uid=0(root) gid=0(root) группы=0(root) контекст=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
[kgkrasnova@kgkrasnova ~]$
```

Рис. 8: Сравнение

Создание и компиляция файла readfile.c (рис. 9).

```
[guest@kgkrasnova ~]$ touch readfile.c
[guest@kgkrasnova ~]$ nano readfile.c
[guest@kgkrasnova ~]$ gcc readfile.c -o readfile
[guest@kgkrasnova ~]$ ls
dir1      readfile.c  simplified2.c  Документы  Музыка      Шаблоны
dir2      simplified  simplified.c  Загрузки  Общедоступные
readfile simplified2  Видео       Изображения  'Рабочий стол'
```

Рис. 9: Создание и компиляция

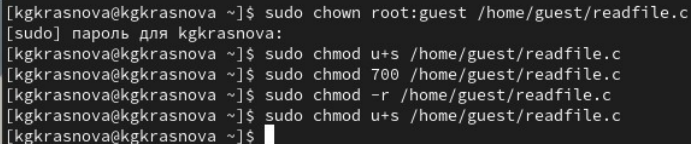
```
С++ Листинг 3 #include <fcntl.h> #include <stdio.h> #include  
<sys/stat.h> #include <sys/types.h> #include <unistd.h> int main (int  
argc, char* argv[]) { unsigned char buffer[16]; size_t bytes_read; int  
i; int fd = open (argv[1], O_RDONLY); do { bytes_read = read (fd,  
buffer, sizeof (buffer)); for (i =0; i < bytes_read; ++i) printf("%c",  
buffer[i]); } while (bytes_read == sizeof (buffer)); close (fd); return  
0; }
```

Содержимое файла (рис. 10).



```
GNU nano 5.6.1                                readfile.c
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int
main (int argc, char* argv[])
{
    unsigned char buffer[16];
    size_t bytes_read;
    int i;
    int fd = open (argv[1], O_RDONLY);
    do
    {
        bytes_read = read (fd, buffer, sizeof (buffer));
        for (i=0; i < bytes_read; ++i) printf("%c", buffer[i]);
    }
    while (bytes_read == sizeof (buffer));
    close (fd);
    return 0;
```

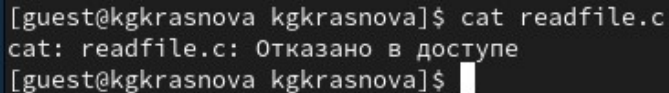
Снова от имени суперпользователя меняю владельца файла readfile. Далее меняю права доступа (рис. 11).

A terminal window with a dark background and light-colored text. It shows a series of commands being executed to change the ownership and permissions of a file named 'readfile.c' located at '/home/guest/'. The commands are: 'sudo chown root:guest /home/guest/readfile.c', followed by a password prompt '[sudo] пароль для kgkrasnova:', then 'sudo chmod u+s /home/guest/readfile.c', 'sudo chmod 700 /home/guest/readfile.c', 'sudo chmod -r /home/guest/readfile.c', and finally 'sudo chmod u+s /home/guest/readfile.c'. The prompt '[kgkrasnova@kgkrasnova ~]\$' is visible at the start of each line.

```
[kgkrasnova@kgkrasnova ~]$ sudo chown root:guest /home/guest/readfile.c
[sudo] пароль для kgkrasnova:
[kgkrasnova@kgkrasnova ~]$ sudo chmod u+s /home/guest/readfile.c
[kgkrasnova@kgkrasnova ~]$ sudo chmod 700 /home/guest/readfile.c
[kgkrasnova@kgkrasnova ~]$ sudo chmod -r /home/guest/readfile.c
[kgkrasnova@kgkrasnova ~]$ sudo chmod u+s /home/guest/readfile.c
[kgkrasnova@kgkrasnova ~]$
```

Рис. 11: Смена прав и владельца

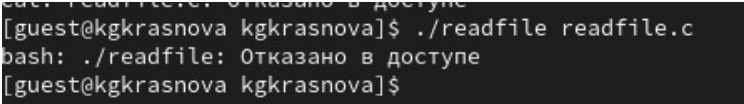
Проверка прочесть файл от имени пользователя guest (рис. 12).

A terminal window with a black background and white text. The prompt is [guest@kgkrasnova kgkrasnova]\$. The user enters the command cat readfile.c. The output is cat: readfile.c: Отказано в доступе. The prompt returns to [guest@kgkrasnova kgkrasnova]\$.

```
[guest@kgkrasnova kgkrasnova]$ cat readfile.c
cat: readfile.c: Отказано в доступе
[guest@kgkrasnova kgkrasnova]$
```

Рис. 12: Проверка

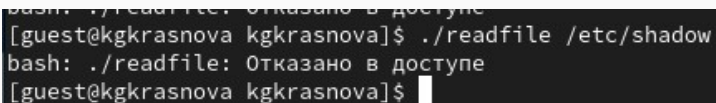
Попытка прочесть тот же файл с помощью программы readfile (рис. 13).



```
cat: readfile.c: Отказано в доступе  
[guest@kgkрасnova kgkрасnova]$ ./readfile readfile.c  
bash: ./readfile: Отказано в доступе  
[guest@kgkрасnova kgkрасnova]$
```

Рис. 13: Отказ

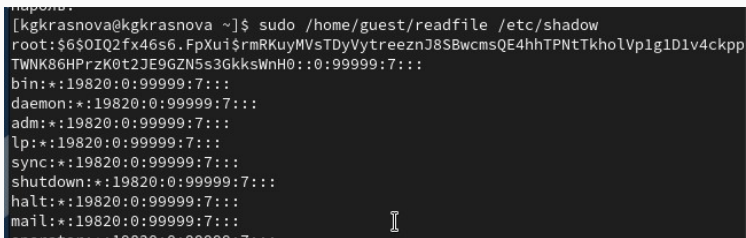
Попытка прочесть файл `\etc\shadow` с помощью программы, все еще получаем отказ в доступе (рис. 14).



```
bash: ./readfile: Отказано в доступе  
[guest@kgkrasnova kgkrasnova]$ ./readfile /etc/shadow  
bash: ./readfile: Отказано в доступе  
[guest@kgkrasnova kgkrasnova]$
```

Рис. 14: Отказ

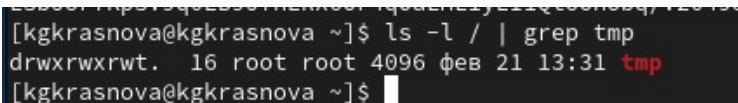
Пробую прочесть эти же файлы от имени суперпользователя и чтение файлов проходит успешно (рис. 15).



```
[kgkrasnova@kgkrasnova ~]$ sudo /home/guest/readfile /etc/shadow
root:$6$0IQ2fx46s6.FpXui$rmRKuyMVSTDyVytreetznJ8SBwcmsQE4hhTPNtTkholVp1g1D1v4ckpp
TWNK86HPrzK0t2JE9GZN5s3GkksWnH0::0:99999:7:::
bin:!:19820:0:99999:7:::
daemon:!:19820:0:99999:7:::
adm:!:19820:0:99999:7:::
lp:!:19820:0:99999:7:::
sync:!:19820:0:99999:7:::
shutdown:!:19820:0:99999:7:::
halt:!:19820:0:99999:7:::
mail:!:19820:0:99999:7:::
operator:!:19820:0:99999:7:::
```

Рис. 15: Суперпользователь

Проверяем папку tmp на наличие атрибута Sticky (рис. 16).



```
[kgkrasnova@kgkrasnova ~]$ ls -l / | grep tmp
drwxrwxrwt. 16 root root 4096 фев 21 13:31 tmp
[kgkrasnova@kgkrasnova ~]$
```

Рис. 16: Проверка

От имени пользователя guest создаю файл с текстом, добавляю права на чтение и запись для других пользователей (рис. 17).

```
[guest@kgkrasnova kgkrasnova]$ echo "test" > /tmp/file01.txt
[guest@kgkrasnova kgkrasnova]$ ls -l /tmp/file01.txt
-rw-r--r--. 1 guest guest 5 фев 21 13:35 /tmp/file01.txt
[guest@kgkrasnova kgkrasnova]$ chmod o+rw /tmp/file01.txt
[guest@kgkrasnova kgkrasnova]$ ls -l /tmp/file01.txt
-rw-r--rw-. 1 guest guest 5 фев 21 13:35 /tmp/file01.txt
[guest@kgkrasnova kgkrasnova]$
```

Рис. 17: Создание файла

Вхожу в систему от имени пользователя guest2, от его имени могу прочитать файл file01.txt, но перезаписать информацию в нем не могу (рис. 18).

```
[kgkrasnova@kgkrasnova ~]$ su guest2
Пароль:
[guest2@kgkrasnova kgkrasnova]$ cat /tmp/file01.txt
test
[guest2@kgkrasnova kgkrasnova]$ echo 'test2' >> /tmp/file01.txt
bash: /tmp/file01.txt: Отказано в доступе
[guest2@kgkrasnova kgkrasnova]$ cat /tmp/file01.txt
test
[guest2@kgkrasnova kgkrasnova]$
```

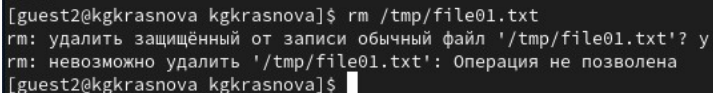
Рис. 18: Попытка перезаписи

Также невозможно добавить в файл file01.txt новую информацию от имени пользователя guest2 (рис. 19).

```
[guest2@kgkrasnova kgkrasnova]$ echo 'test3' > /tmp/file01.txt
bash: /tmp/file01.txt: Отказано в доступе
[guest2@kgkrasnova kgkrasnova]$ cat /tmp/file01.txt
test
[guest2@kgkrasnova kgkrasnova]$
```

Рис. 19: Добавление инф-и

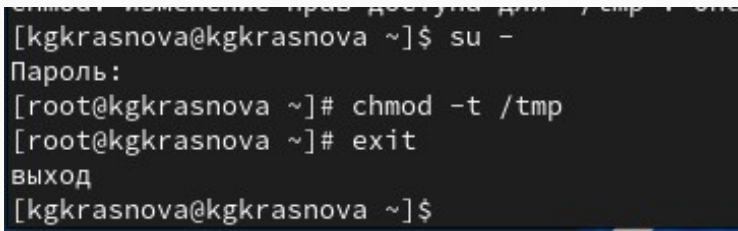
Далее пробуем удалить файл, снова получаем отказ (рис. 20).

A terminal window with a dark background and light-colored text. The prompt is [guest2@kgkrasnova kgkrasnova]\$. The command rm /tmp/file01.txt is entered. The output shows two lines of error messages: 'rm: удалить защищённый от записи обычный файл '/tmp/file01.txt'? y' and 'rm: невозможно удалить '/tmp/file01.txt': Операция не позволена'. The prompt is repeated at the end of the second line, followed by a cursor.

```
[guest2@kgkrasnova kgkrasnova]$ rm /tmp/file01.txt
rm: удалить защищённый от записи обычный файл '/tmp/file01.txt'? y
rm: невозможно удалить '/tmp/file01.txt': Операция не позволена
[guest2@kgkrasnova kgkrasnova]$
```

Рис. 20: Удаление

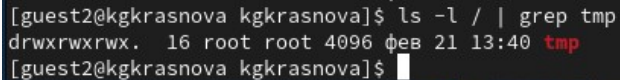
От имени суперпользователя снимаем с директории атрибут Sticky (рис. 21).



```
kgkраснова@kgkраснова ~]$ su -  
Пароль:  
[root@kgkраснова ~]# chmod -t /tmp  
[root@kgkраснова ~]# exit  
выход  
kgkраснова@kgkраснова ~]$
```

Рис. 21: Снятие атрибута

Проверяем, что атрибут действительно снят (рис. 22).



```
[guest2@kgkrasnova kgkrasnova]$ ls -l / | grep tmp
drwxrwxrwx. 16 root root 4096 фев 21 13:40 tmp
[guest2@kgkrasnova kgkrasnova]$
```

Рис. 22: Проверка

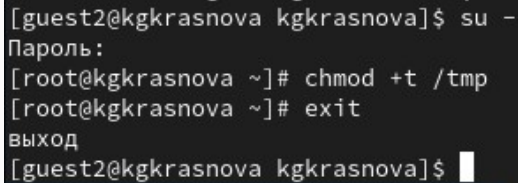
Выполнение лабораторной работы

Далее был выполнен повтор предыдущих действий (рис. 23).

```

[guest2@kgkrasnova kgkrasnova]$ echo 'test2' >> /tmp/file01.txt
bash: /tmp/file01.txt: Отказано в доступе
[guest2@kgkrasnova kgkrasnova]$ cat /tmp/file01.txt
test
[guest2@kgkrasnova kgkrasnova]$ echo 'test3' > /tmp/file01.txt
bash: /tmp/file01.txt: Отказано в доступе
[guest2@kgkrasnova kgkrasnova]$ cat /tmp/file01.txt
test
[guest2@kgkrasnova kgkrasnova]$ rm /tmp/file01.txt
rm: удалить защищённый от записи обычный файл '/tmp/file01.txt'? y
[guest2@kgkrasnova kgkrasnova]$ ls -l / | grep tmp
drwxrwxrwx. 16 root root 4096 фев 21 13:42 tmp
[guest2@kgkrasnova kgkrasnova]$ ls -l /home/guest
итого 72
drwxr-xr-x. 2 root root 18 фев 21 09:42 dir1
drwxr-xr-x. 2 guest guest 19 фев 21 10:55 dir2
-rwxr-xr-x. 1 guest guest 17600 фев 21 13:26 readfile
--ws-----. 1 root guest 401 фев 21 13:25 readfile.c
-rwxr-xr-x. 1 guest guest 17552 фев 21 13:05 simplified
-rwsr-xr-x. 1 root guest 17656 фев 21 13:11 simplified2
-rw-r--r--. 1 guest guest 302 фев 21 13:11 simplified2.c
-rw-r--r--. 1 guest guest 175 фев 21 13:05 simplified.c
drwxrwxr-x. 2 guest guest 6 фев 14 12:58 Видео
drwxrwxr-x. 2 guest guest 6 фев 14 12:58 Видео
```

Возвращение директории tmp атрибута t от имени суперпользователя (рис. 24).

A terminal window with a black background and white text. The prompt is [guest2@kgkrasnova kgkrasnova]\$. The user enters 'su -'. The prompt changes to [root@kgkrasnova ~]#. The user enters 'chmod +t /tmp'. The prompt remains [root@kgkrasnova ~]#. The user enters 'exit'. The prompt returns to [guest2@kgkrasnova kgkrasnova]\$. The word 'выход' is written on the line before the final prompt. A white cursor is visible at the end of the final prompt line.

```
[guest2@kgkrasnova kgkrasnova]$ su -  
Пароль:  
[root@kgkrasnova ~]# chmod +t /tmp  
[root@kgkrasnova ~]# exit  
выход  
[guest2@kgkrasnova kgkrasnova]$
```

Рис. 24: Возвращение

Изучила механизм изменения идентификаторов, применила SetUID- и Sticky-биты. Получила практические навыки работы в кон- соли с дополнительными атрибутами. Рассмотрела работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.